第六节 打包解包及其关键技术

肖逸飞

xyf1989@uestc.edu.cn

课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. 打包解包及其关键技术
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（满分10分）：支持特定文件系统的特殊文件（管道/链接/设备等）

元数据支持（满分10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各3分）：支持筛选需要备份的文件（路径/类型/名字/时间/尺寸/用户）

打包解包（每种算法10分）：将所有备份文件拼接为一个大文件保存

压缩解压（每种算法10分）：通过文件压缩节省备份文件的存储空间

加密解密（每种算法10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（满分10分）：实现友好易用的GUI界面

定时备份（满分10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（满分15分）：自动感知用户文件变化，进行自动备份

网络备份（分值具体而论）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（满分10分）：支持特定文件系统的特殊文件（管道/链接/设备等）

元数据支持（满分10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各3分）：支持筛选需要备份的文件（路径/类型/名字/时间/尺寸/用户）

打包解包（每种算法10分）：将所有备份文件拼接为一个大文件保存

压缩解压（每种算法10分）：通过文件压缩节省备份文件的存储空间

加密解密（每种算法10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（满分10分）：实现友好易用的GUI界面

定时备份（满分10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（满分15分）：自动感知用户文件变化，进行自动备份

网络备份（分值具体而论）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

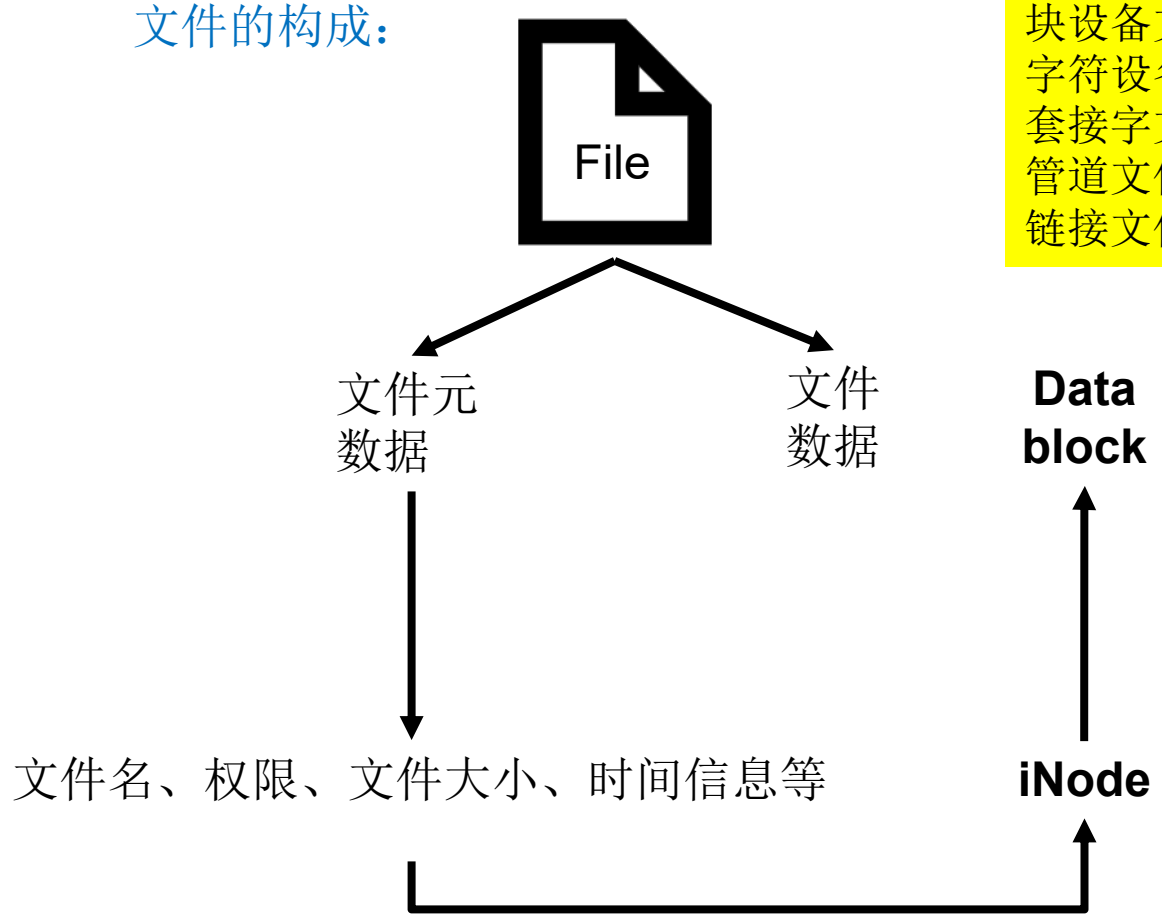
UNIX 文件系统原理

Unix文件系统

在Unix系统中，有一句话：一切皆文件!

普通文件
目录文件
块设备文件（/dev/sda1,/dev/sda2…）
字符设备文件（串口设备，如键盘、鼠标）
套接字文件（网络通信）
管道文件（进程间通信，有名管道、无名管道）
链接文件（软连接、硬链接）

文件的构成：

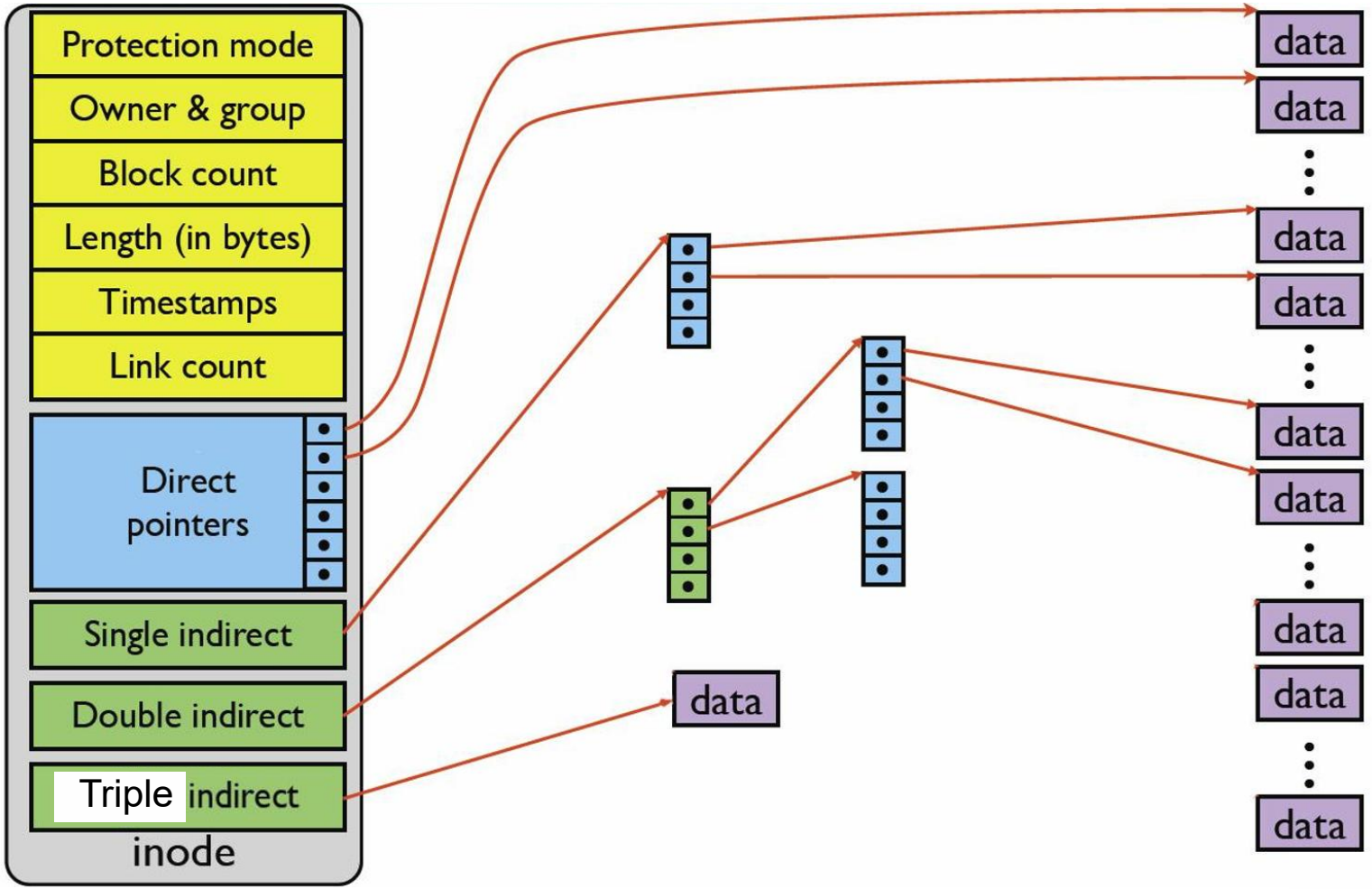


思考：文件的路径信息存储在哪里？

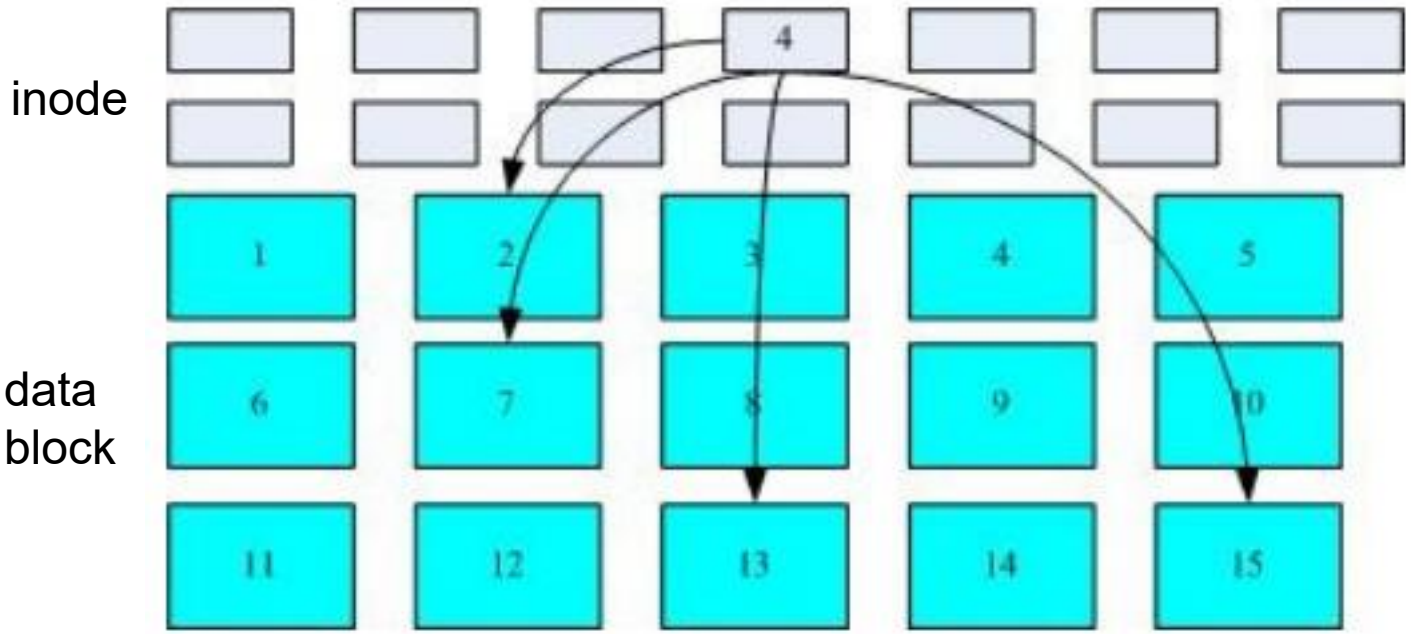
Unix文件系统

普通文件

inode: index node(索引节点)



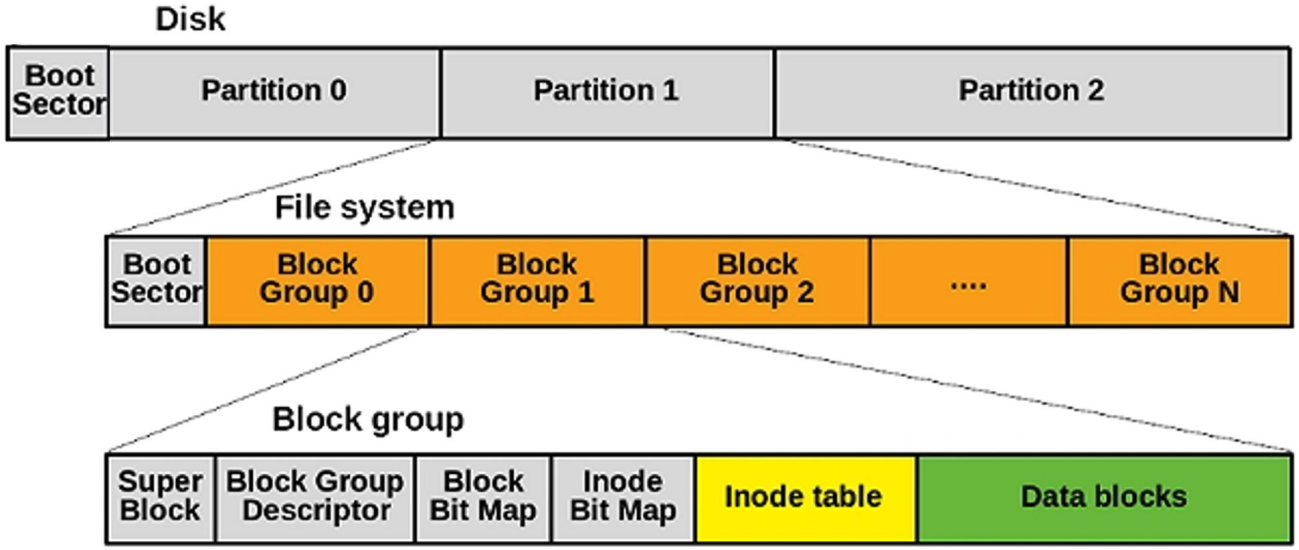
Unix文件系统



Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）

ext2 文件系统的基本思想是将整个磁盘分区（例如 /dev/sda1）等量划分为多个更小、更易管理的单元，称为**块组（Block Group）**。



引导块：位于整个分区最开始的第一个块（扇区）。它不属于任何块组，用于存储系统的引导程序。如果该分区不是启动分区，这个块可能为空。

块组：分区剩余的空间被顺序地划分为 N 个块组。每个块组的结构都几乎完全相同。这种设计有两个主要优点：

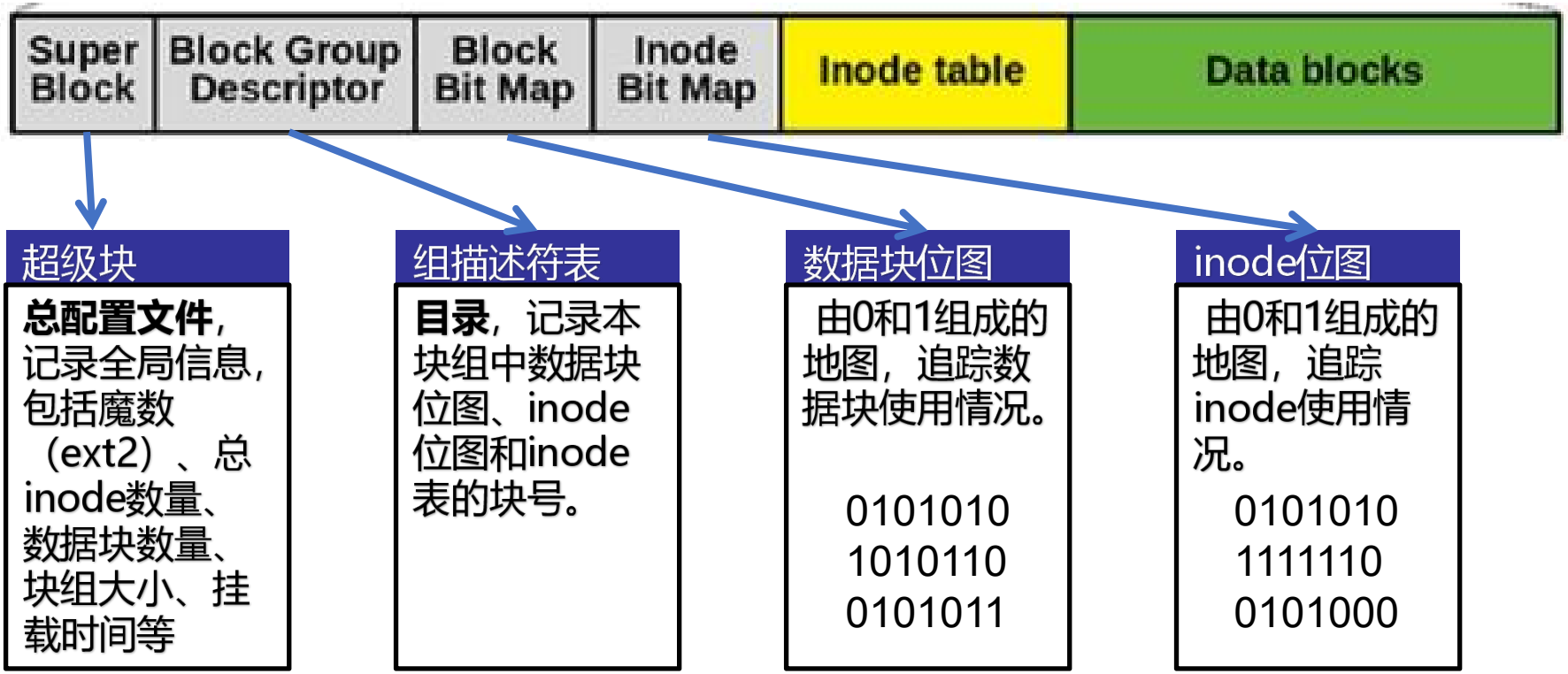
- 性能：将相关的 inode 和数据块放在一起，减少磁头寻道时间（针对机械硬盘）。
- 可靠性：关键元数据（如超级块）在每个块组都有备份，一旦某个备份损坏，可以从其他备份中恢复。

ext2 官方文档：https://www.nongnu.org/ext2-doc/ext2.html

Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）

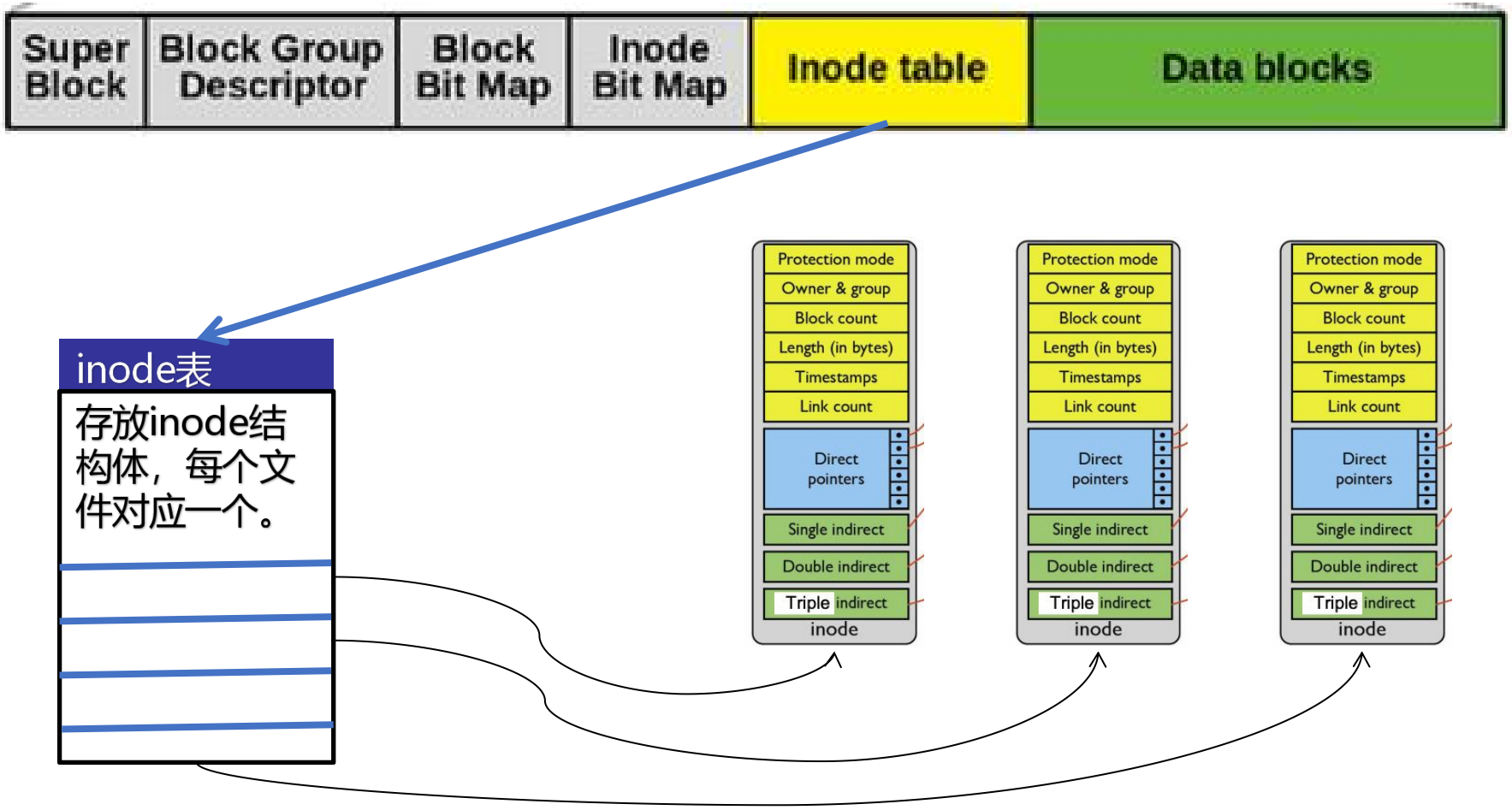
块组：



ext2 官方文档：https://www.nongnu.org/ext2-doc/ext2.html

Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）



ext2 官方文档：https://www.nongnu.org/ext2-doc/ext2.html

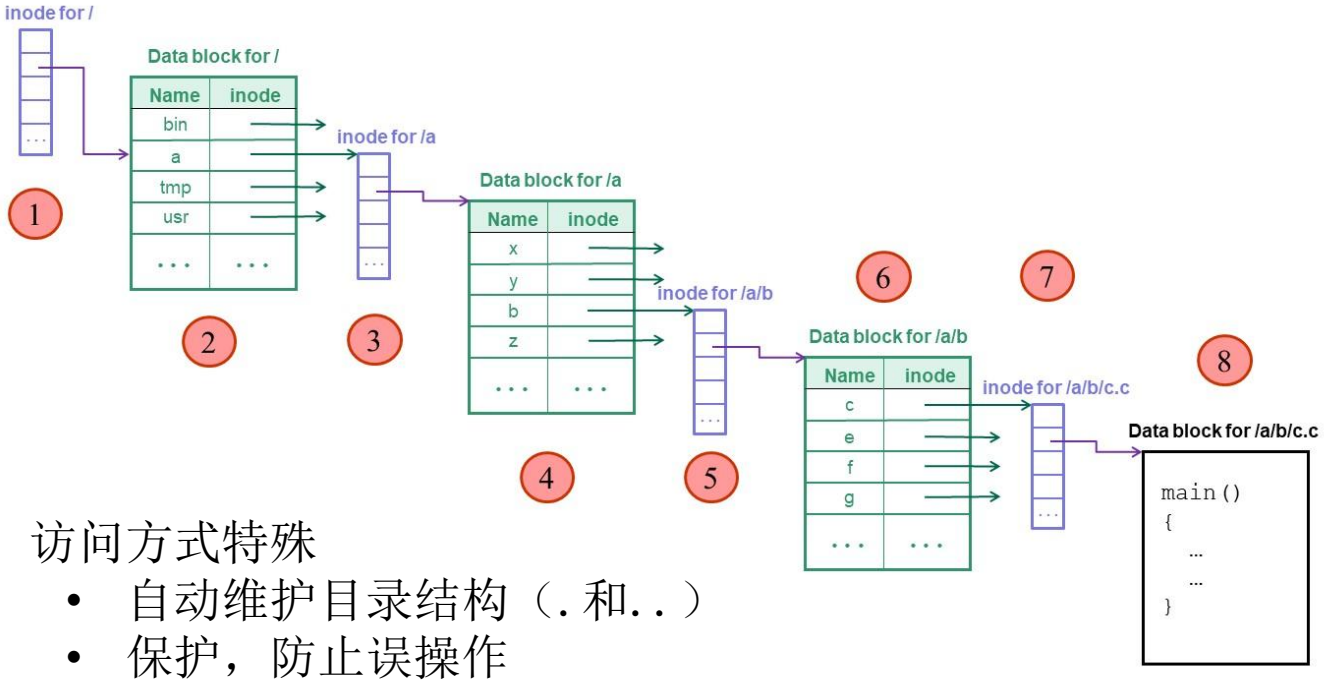
思考：文件的路径信息存储在哪里？

Unix文件系统

目录文件的实现

- 目录也是文件！
  - 有特殊结构的文件
  - 以访问 /a/b/c.c 为例

文件的路径其实没有进行存储，只是一个字符串匹配（寻路）的过程

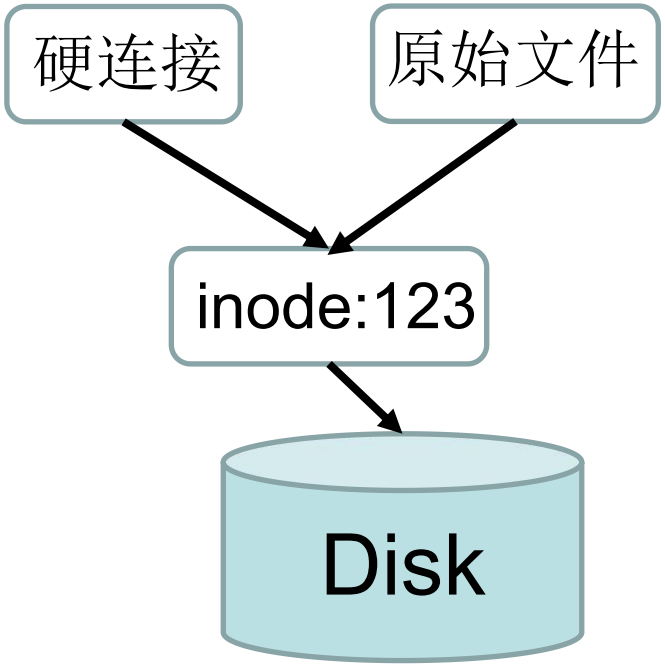


- 访问方式特殊
  - 自动维护目录结构（.和..）
  - 保护，防止误操作

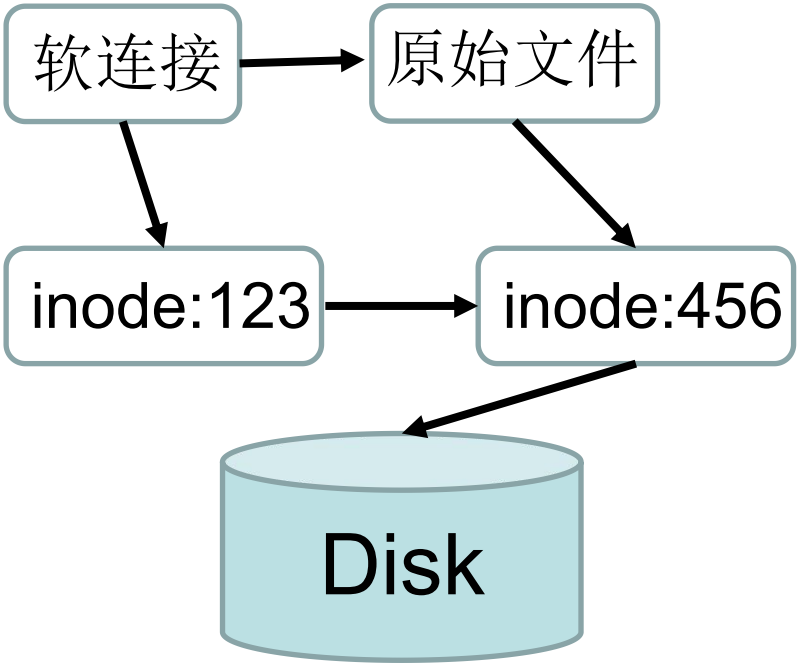
思考：如果两个文件同时指向同一个inode，是什么？

Unix文件系统

文件链接（软连接、硬链接）



指向同一个inode，inode的引用计数+1

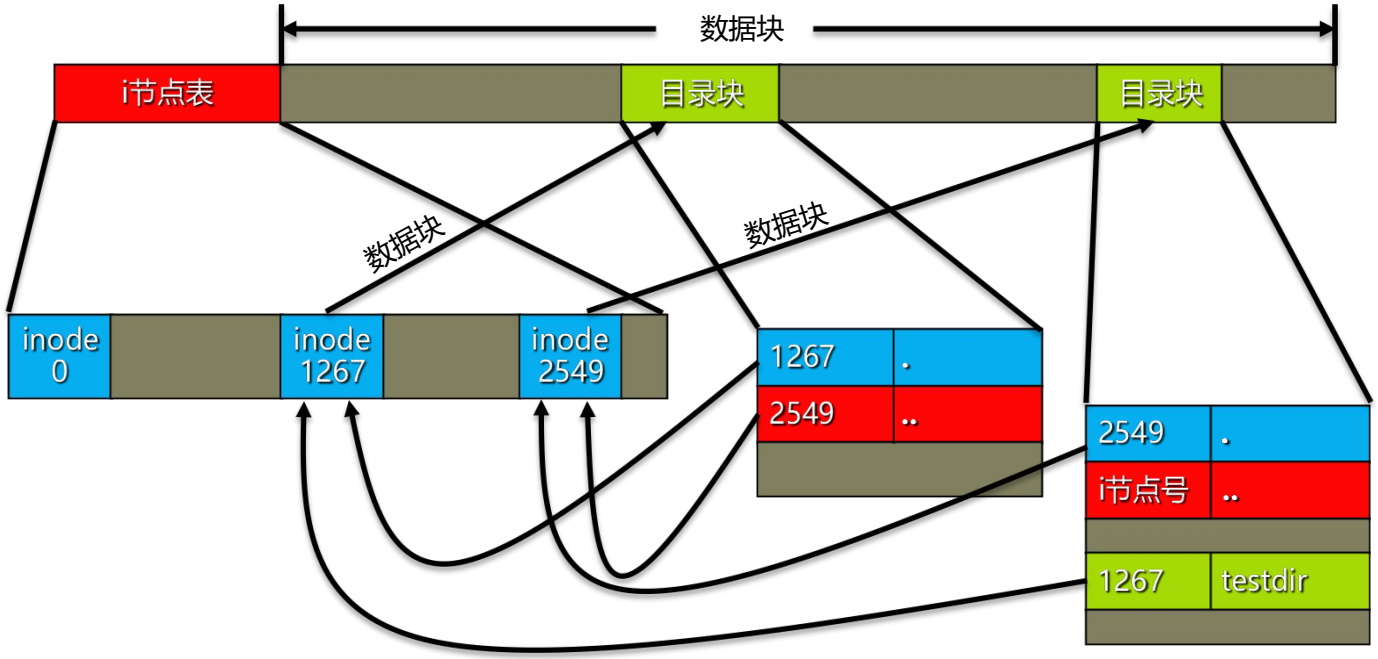


有自己的inode，文件内容记录了目标文件的路径。

思考：为什么新建一个文件夹，其引用数为2？

Unix文件系统

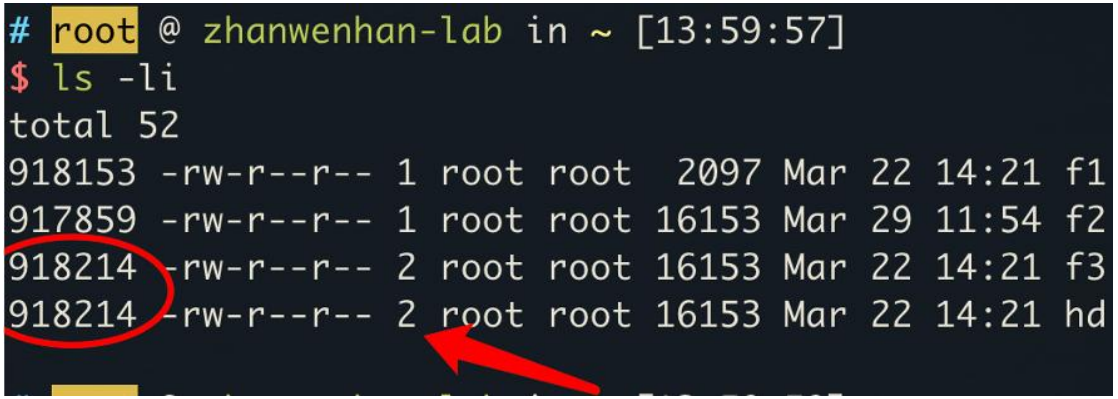
例：指向父子目录中目录项之间的关系。



Unix文件系统

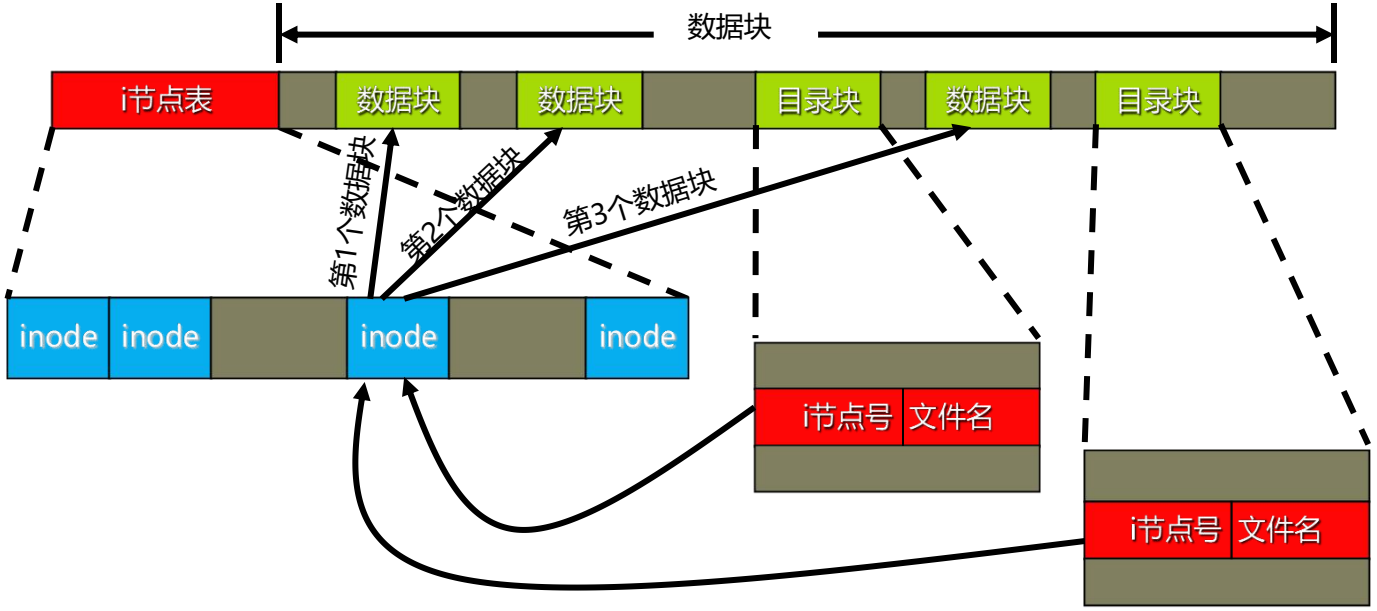
链接数（硬链接）

- 指向同一个 inode 的目录项的数量
- 硬链接实现的是 inode 共享
- ./ 和 ../ 会影响到链接数



Unix文件系统

例：指向同一个 inode 的两个目录项。



模拟cp -a或tar进行打包和解包

cp -a 对不同文件类型的处理

1. 普通文件 (Regular File)
行为：复制文件内容+元数据（权限、时间戳等）
实现：读取内容，创建新文件，写入内容，设置元数据

2. 目录文件 (Directory)
行为：递归复制整个目录树
实现：创建目录，然后递归处理目录内容

3. 符号链接 (Symbolic Link)
行为：复制链接本身，而不是链接指向的文件
实现：读取链接目标，创建新的符号链接

cp -a 对不同文件类型的处理

4. 硬链接 (Hard Link)
行为：复制为独立的文件（失去硬链接关系）
特殊情况：如果使用 -l 或 --link 选项会尝试保持硬链接

有什么问题？

5. 设备文件 (Device File)
字符设备/块设备：创建新的设备节点（相同的设备号）
实现：使用 mknod() 系统调用，通常需要root权限

6. FIFO/命名管道 (Named Pipe)
行为：创建新的 FIFO 文件
实现：使用 mkfifo() 系统调用

7. 套接字 (Socket)
行为：通常无法复制，会报错或跳过
原因：套接字是活跃的连接端点，复制没有意义

Tar

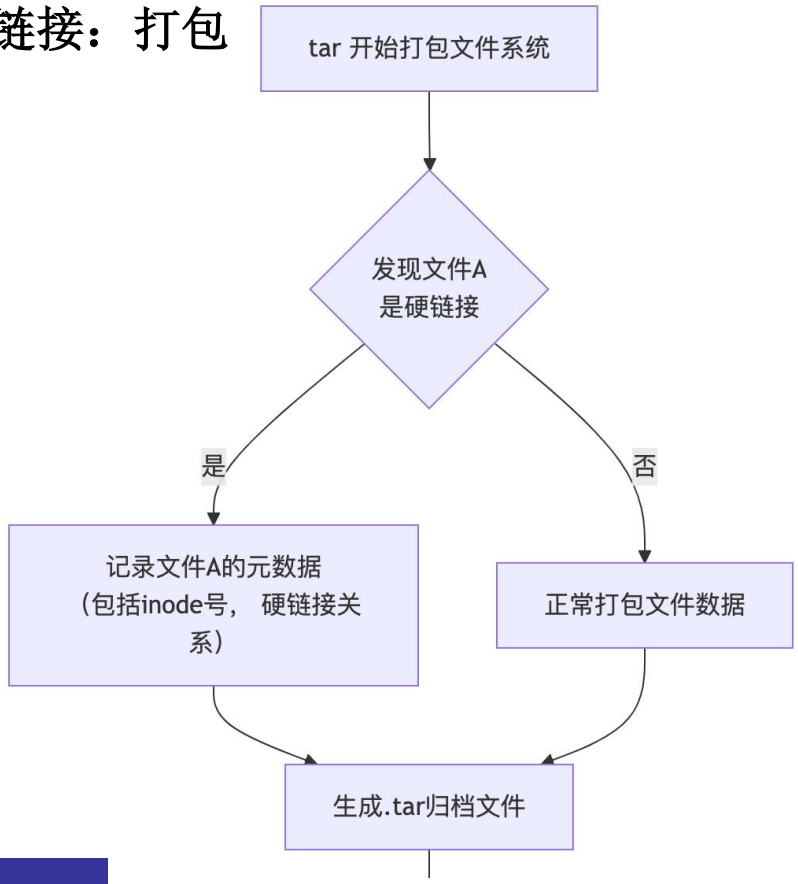
Unix文件系统

文件类型	打包行为	解包行为	备注
普通文件	将文件内容和元数据（权限、时间戳等）记录到归档中。	在目标位置创建新文件，并写入原内容，恢复元数据。	最基础的处理方式。
目录	递归地进入目录，将其下的所有文件和子目录打包。	创建目录，并恢复其权限等属性，然后解包其中的内容。	保持整个目录结构的基石。
符号链接	默认仅保存链接本身的路径（即指向哪里），不跟随链接去打包目标文件的内容。	在目标位置重新创建符号链接，指向记录的路径。	若解包时目标路径不存在，链接会变成“悬空链接”（dangling link）。
硬链接	识别到多个路径指向同一个 Inode 时，会保存这种硬链接关系，而非文件数据的多个副本。	在目标位置重新创建硬链接，确保多个文件名链接到同一个数据块。	高效节省空间，避免重复存储。是保持文件系统内部关联的重要机制。
设备文件（字符/块设备）	保存设备文件的类型和主/次设备号等元数据。	尝试根据记录的设备号创建设备文件。	通常需要 root 权限才能成功创建。归档时一般不包含 /dev目录下的设备文件。
FIFO（命名管道）	保存其文件类型信息。	创建新的 FIFO 文件。	

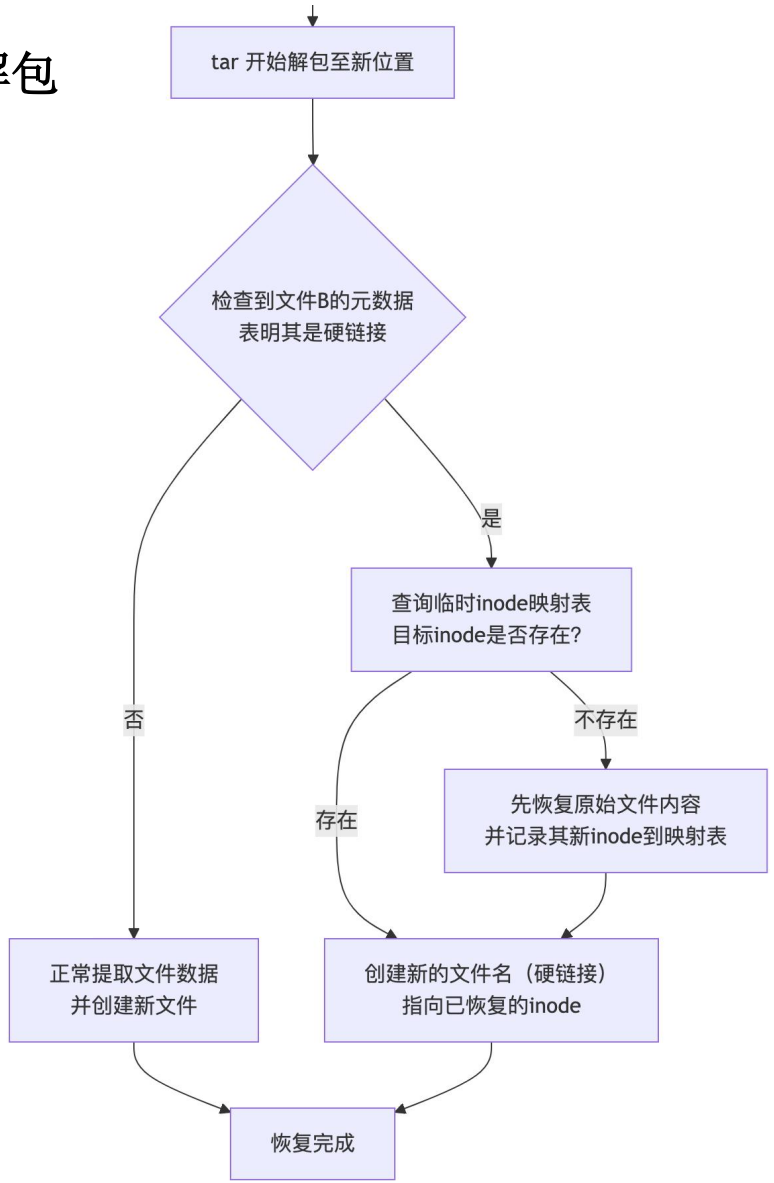
Unix文件系统

Tar

硬链接：打包



解包



cp -a

硬链接 == 普通文件

文件遍历&打包解包

打包解包原理

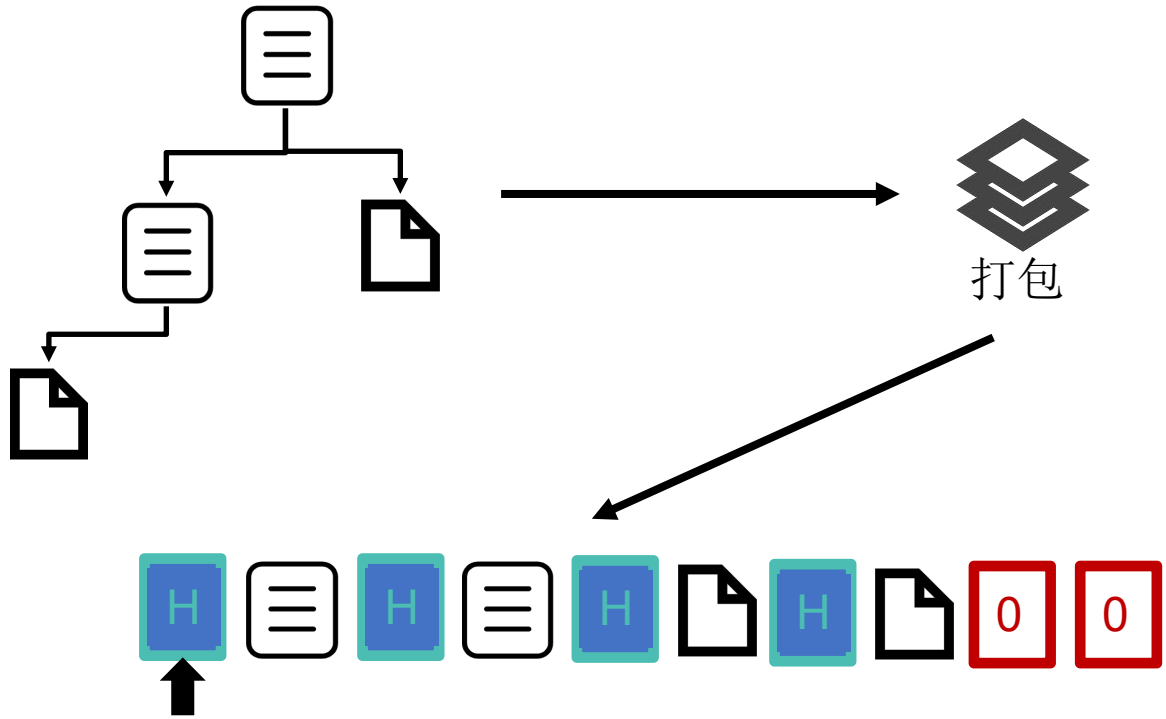
文件遍历方法

文件打包

打包：将多个文件/目录整合为一个大文件的过程。

解包：将多个文件/目录从一个大文件中按原样恢复的过程。

大部分压缩软件支持同时对文件打包。最经典的文件打包程序：Tar。

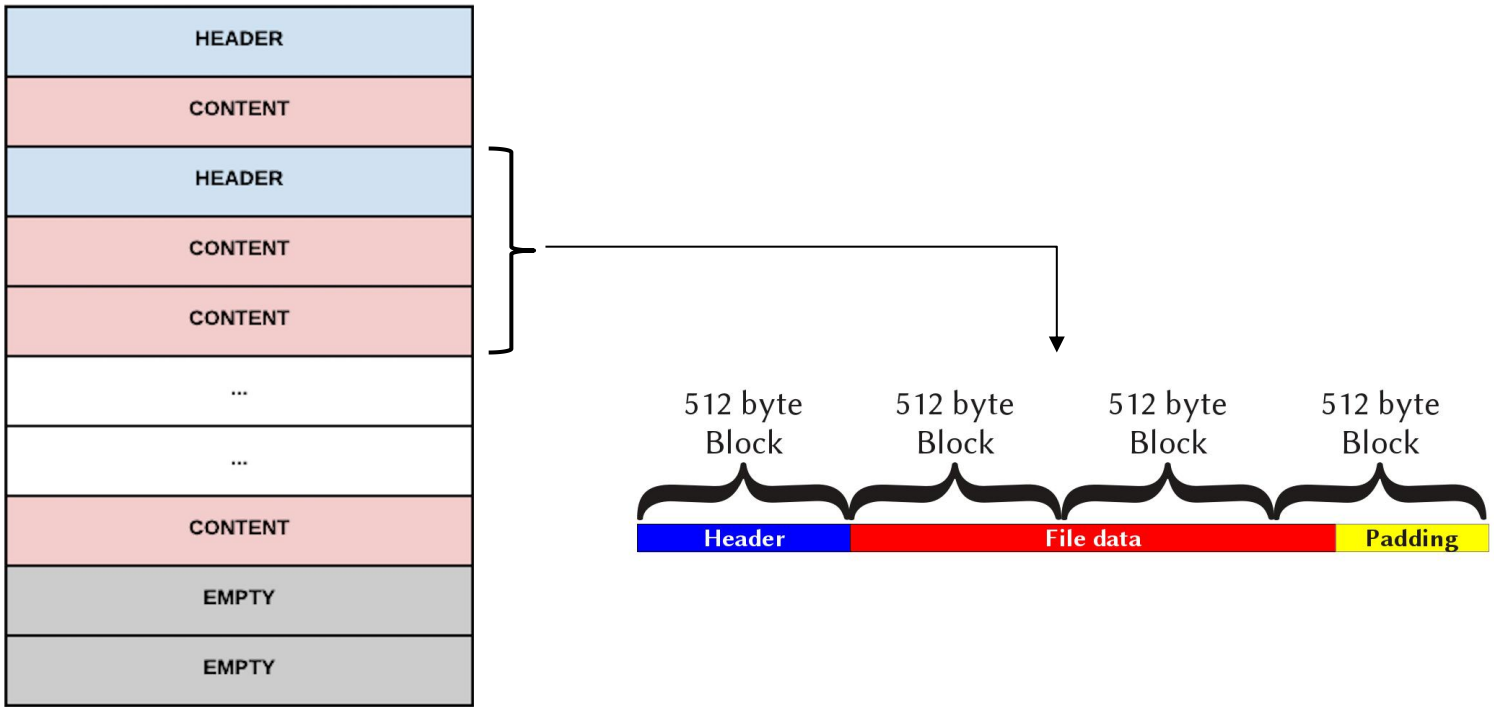


文件名、权限、路径、文件大小、时间信息等

Tar打包原理

Tar是Unix和类Unix系统上的归档打包工具，可以将多个文件合并为一个文件，打包后的文件名亦为“tar”。目前，tar文件格式已经成为POSIX标准。

Tar归档文件由一系列文件对象通过线性排列的方式组合而成。每个文件对象都包含一个或者若干个512字节的记录块，并以一个头记录开头。文件数据按原样写入，但其长度舍入为512字节的倍数。



Tar打包原理

tar一共经历了3个阶段：

Pre-POSIX.1-1988：1988年以前，Pre-POSIX.1-1988(i.e. v7)的tar虽然被广泛应用，但是存在文件名过短的问题（只有100字节，还包括路径前缀）。同时，虽然有12B用于记录文件大小，但是由于历史原因，仅采用11个8进制数字保存文件大小，只能记录最大8GB的文件。

POSIX.1-1988（UStar）：1988年，UNIX的UStar被广泛采用，即POSIX.1-1988，它解决了文件名过短的问题，因为该版本可以允许256字节用于存放文件路径及文件名（256B=prefix(155)+filename(100)）。

POSIX.1-2001（pax）：1997年，SUN公司提出了新的方法，后被引入到了POSIX.1-2001，即extened tar 或 pax。这种格式支持无限制的文件名，无限制的软连接命名，用户名和组名无限制，文件大小无限制，uid和gid也无限制。该版本也是目前主流tar软件采用的格式。

```
union {
   union { // Pre-POSIX.1-1988 format
      struct {
         char name[100]; // file name
         char mode[8]; // permissions
         char uid[8]; // user id (octal)
         char gid[8]; // group id (octal)
         char size[12]; // size (octal)
         char mtime[12]; // modification time (octal)
         char check[8]; // sum of unsigned characters in block,
               with spaces in the check field while calculation is done (octal)
         char link; // link indicator
         char link_name[100]; // name of linked file
       };
       // UStar format (POSIX IEEE P1003.1)
       struct {
         char old[156]; // first 156 octets of Pre-POSIX.1-1988 format
         char type; // file type
         char also_link_name[100]; // name of linked file
         char ustar[8]; // ustar\000
         char owner[32]; // user name (string)
         char group[32]; // group name (string)
         char major[8]; // device major number
         char minor[8]; // device minor number
         char prefix[155];
       };
   };
   char block[512]; // raw memory (500 octets of actual data, padded to 1 block)
};
```

文件名

文件大小

```
#define REGULAR          0
#define NORMAL          '0'
#define HARDLINK        '1'
#define SYMLINK         '2'
#define CHAR            '3'
#define BLOCK           '4'
#define DIRECTORY       '5'
#define FIFO            '6'
```

文件类型

文件路径

文件内容

实现思路

- 将**目录遍历算法**和**打包算法**分开实现和执行
  - 目录遍历算法：
    - 输入：某个目录
    - 输出：目录下的所有文件（全部类型）列表
  - 打包算法：
    - 输入：文件列表
    - 输出：打包文件
  - 解包算法：
    - 输入：打包文件，解包目录
    - 输出：解包数据

实现思路

- 将**目录遍历算法**和**打包算法**分开实现的优点和缺点
  - 优点：
    - 便于逻辑解耦
    - 易于实现
    - 便于实现复杂的文件过滤逻辑
  - 缺点：
    - 时间效率和空间效率下降
    - 打包过程中的数据一致性可能会出现问题
  - 目录遍历算法输出文件列表的顺序对解包算法很重要
    - 结论：先根遍历

实现思路

- 实现**目录遍历算法**应该注意哪些问题
  - 硬连接如何处理？
    - 直接复制 or 保持链接关系？
  - 软连接如何处理？
    - 直接复制 or 仅复制链接？
    - 目录软链接可能导致遍历死循环（lstat）
  - 相对路径 or 绝对路径？
    - 建议将程序的当前路径调整为对应目录再开始遍历
  - 如何进行数据过滤？
    - 在遍历过程中对数据进行筛选
    - 仅保存和继续遍历通过筛选的数据

实现思路

- 实现打包算法应该注意哪些问题
  - 输入安全性检查
    - 源路径存在性，可访问性
    - 目的路径存在性，可访问性
  - 程序当前路径要和遍历时的当前路径一致
  - 文件类型的支持
    - 普通文件
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
  - 文件名超长处理
    - 使用多个header块（如何标识？）

实现思路

- 实现解包算法应该注意哪些问题
  - 输入安全性检查
    - 源文件存在性，可访问性
    - 源文件头校验
    - 目的路径存在性，可访问性
    - 目的路径目录为空！
  - 文件恢复
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
    - 普通文件（包括数据恢复，注意文件大小）
  - 元数据的恢复
  - 文件名超长处理
    - 使用多个header块（如何确认）

参考文件遍历&打包解包代码

Unix文件系统

如何保证备份软件的正确性：

1、文件类型
2、文件路径
3、文件数据
4、文件元数据（属主/权限/时间）
5、文件链接方式（软/硬）

Unix文件系统

项目涉及到的系统API（部分）：

- open
- close
- read
- write
- stat
- chown
- chmod
- utimes
- opendir
- closedir
- readdir
- rewinddir
- mkdir
- mkfifo
- unlink
- link
- symlink
- ……

man手册



严禁使用cp -a命令

实验环节

完善数据备份软件的：

框架代码

完成数据备份软件的：

目录遍历部分

文件过滤部分

打包解包部分

（推荐）完成单元测试